

整合練習 2：怎麼找 BAPI (實案：QA12 檢驗批過帳)

Lecture

if01 講了 BAPI 的六個判斷條件，但那是「已經知道答案」的情況下做的整理。真實工作情境通常是反過來的：你知道一個交易碼 / 一個業務動作，但不知道背後該呼叫哪個介面。這題用 **QA12 (變更使用決策 / Usage Decision)** 當實案，示範怎麼從「一個畫面」摸到「該呼叫的程式介面」，而且特意選了一個「答案不只一個」的情境——因為現實中很多時候「該用哪個介面」本身就要先分清楚場景。

先講清楚要解決的兩種場景：

- **情境 A**——下使用決策的「同時」要自動過帳：檢驗員在 QA12 畫面上直接勾選/設定「同時過帳」，一次做完判定跟庫存異動
- **情境 B**——判定已經下了，但庫存還沒過帳 (呼應 SAP 標準的「Post Open Stocks」情境：SAP Help 文件 *Post Open Stocks (QM-IM-UD)* 描述這個狀況——貨還卡在檢驗庫存，系統甚至會觸發 Workflow 提醒負責人補過帳)，需要另外單獨補一次過帳動作

這兩種場景在 SAP 標準系統裡不是同一支程式處理，這正是這題的核心教訓。

情境 A：BAPI_INSPLLOT_SETUSAGEDECISION

用本課程 [.claude/rules/sap-adt-mcp.md](#) 第 2 節記載的 ADT quickSearch workaround (比 BAPI Explorer / SE37 手動點還快)，直接在這套連線系統 (client 130) 查證，**BAPI_INSPLLOT_*** 這一整組 BAPI 都存在，掛在 **Function Group 2045** (跟 Business Object BUS2045 對上了)、Package **QL**：

BAPI_INSPLLOT_GETDETAIL	Function Module	套件 QL
BAPI_INSPLLOT_GETLIST	Function Module	套件 QL
BAPI_INSPLLOT_GETOPERATIONS	Function Module	套件 QL
BAPI_INSPLLOT_GETSTATUS	Function Module	套件 QL
BAPI_INSPLLOT_SETUSAGEDECISION	Function Module	套件 QL
BAPI_INSPLLOT_STATINTERFACE	Function Module	套件 QESI (另一個 Function Group QIST)

BAPI_INSPLLOT_SETUSAGEDECISION 的真實介面 (`sap_get_source` 直接讀出來的，不是猜的)：

```
FUNCTION bapi_insplot_setusagedecision
  IMPORTING
    VALUE(number) LIKE bapi2045ud-insplot
    VALUE(ud_data) LIKE bapi2045ud
    VALUE(language) LIKE bapi2045la OPTIONAL
    VALUE(ud_multspec) LIKE bapi2045s1-ind_x_or_blank OPTIONAL
    VALUE(ud_mode) TYPE bapi2045d_il4-ud_mode DEFAULT 'D'
  EXPORTING
    VALUE(ud_return_data) LIKE bapi2045ud_return
    VALUE(stock_data) LIKE bapi2045d_il2
    VALUE(return) LIKE bapireturn1
```

```
VALUE(/cwm/stock_data) TYPE /cwm/bapi2045d_il2
TABLES
  system_status LIKE bapi2045ss OPTIONAL
  user_status LIKE bapi2045us OPTIONAL.
```

原始碼開頭就有一段**清楚寫明呼叫慣例的註解**（這是 if03 最重要的第一手證據，不是推測）：

```
* Attention:
* In case this BAPI was called successfully (Message 050, ID 'QV')    --*
* --> BAPI 'BAPI_TRANSACTION_COMMIT' has to be called                --*
*     for saving the usage decision data.                             --*
* otherwise (see structure return for message)                       --*
* --> BAPI 'BAPI_TRANSACTION_ROLLBACK' has to be called              --*
*     to e.g. dequeue the locked data                                 --*
```

再往下讀，**ud_data-ud_stock_posting** 這個欄位會被原封不動傳進內部呼叫的 **QEVC_PROCESS_AUTO_UD**：

```
CALL FUNCTION 'QEVC_PROCESS_AUTO_UD'
EXPORTING
  ...
  i_stock_posting      = ud_data-ud_stock_posting
  ...
```

這證實了「同時過帳」是靠 **ud_data-ud_stock_posting = 'X'** 這一個旗標觸發的——BAPI 介面本身沒有另外開一個「要不要過帳」的獨立參數，是塞在 **ud_data**（型別 **BAPI2045UD**）這個結構裡的一個欄位。

BAPI2045UD 的關鍵欄位（**sap_get_source** 讀 DDIC 結構得到）：

欄位	型別	意義
insplot	QPLOS	檢驗批號
ud_selected_set	QVMENGE	使用決策的選用集合（Selected Set）
ud_plant	WERKS_D	工廠
ud_code_group	QVGRUPPE	決策代碼群組
ud_code	QVCODE	決策代碼（如 A = Accept）
ud_text_line	QEIFVETEXT	決策備註文字
ud_force_completion	QINSP_CAN	強制完成檢驗
ud_stock_posting	QSTOCK_PST	是否同時過帳的旗標——情境 A 的關鍵欄位

情境 B：找不到公開 BAPI，往回追出四支底層 FM

情境 B（單純補過帳，**BAPI_INSLOT_SETUSAGEDECISION** 不適用）沒有對應的公開 BAPI——用同一招 ADT quickSearch 直接指名搜尋，這次不是靠猜命名規則，而是拿使用者已經知道的 FM 名稱去確認它們**確實存在於這套系統**：

MB_CREATE_GOODS_MOVEMENT	Function Group MBWL	套件 MB
MB_POST_GOODS_MOVEMENT	Function Group MBWL	套件 MB
QAMB_COLLECT_RECORD	Function Group QAMB	套件 QL
STATUS_CHANGE_INTERN	Function Group BSVA	套件 BSV

四支都在，四個不同 Function Group、三個不同套件——這本身就說明了「情境 B 沒有一個統一的公開介面」，是東拼西湊出來的標準內部呼叫序列。逐一看重點：

① **MB_CREATE_GOODS_MOVEMENT + MB_POST_GOODS_MOVEMENT**——這兩支不是用參數溝通，是共用 **Function Group** 全域記憶體：

```
FUNCTION mb_create_goods_movement
IMPORTING
  VALUE(imkpf) LIKE imkpf
  ...
EXPORTING
  VALUE(emkpf) LIKE emkpf
  ...
TABLES
  emseg LIKE emseg
  imseg LIKE imseg
  ...
```

```
FUNCTION mb_post_goods_movement
IMPORTING
  VALUE(xblnr_sd) TYPE any OPTIONAL
EXPORTING
  VALUE(emkpf) LIKE emkpf.
  ...
READ TABLE xmseg INDEX 1.
IF NOT sy-subrc IS INITIAL.
  MESSAGE a260.
ENDIF.
IF xmkpf-mblnr IS INITIAL.
  MESSAGE a260.
ENDIF.
```

注意 **MB_POST_GOODS_MOVEMENT** 的介面幾乎是空的（只有一個 **xblnr_sd** 可選參數），但方法本體卻直接讀取 **xmkpf** / **xmseg** 這兩個變數——這兩個變數不是這支 FM 自己宣告的區域變數，是 **MBWL** 這個 **Function Group** 的全域資料（**TOP include** 宣告），**MB_CREATE_GOODS_MOVEMENT** 執行時會把組好的憑證資料寫進這些全域變數，**MB_POST_GOODS_MOVEMENT** 再讀出來寫入資料庫。這是這題最重要的架構發現：這兩支 FM 必須在同一個 **Function Group** 實例（同一次呼叫的同一個 **session**）依序呼叫，不能拆到不同的 RFC 連線或不同批次分別呼叫——這也是它們沒有做成正式 BAPI（BAPI 講究每支介面自己帶齊所有輸入輸出、不依賴外部共享狀態）的根本原因。

② **QAMB_COLLECT_RECORD**——不是「呼叫了立刻寫進 **QAMB**」，是「先塞進全域緩衝表，**COMMIT** 時才真正寫」：

```
FUNCTION QAMB_COLLECT_RECORD
  IMPORTING
    VALUE(LOTNUMBER) LIKE QALS-PRUEFLOS
    VALUE(DOCYEAR) LIKE MSEG-MJAHR
    VALUE(DOCNUMBER) LIKE MSEG-MBLNR
    VALUE(DOCPOSITION) LIKE MSEG-ZEILE OPTIONAL
    VALUE(TYPE) LIKE QAMB-TYP
    VALUE(REFRESH) LIKE QM00-QKZ OPTIONAL.

  ...
  INSERT L_QAMB INTO G_QAMBTAB INDEX L_TABIX.
  ...
  PERFORM SICHERN_QAMB ON COMMIT.
ENDFUNCTION.

FORM SICHERN_QAMB.
  ...
  CALL FUNCTION 'QEVA_MATERIALDOC_TO_LOT' IN UPDATE TASK
    TABLES
      T_QAMBTAB = G_QAMBTAB.
ENDFORM.
```

介面吃的是「檢驗批號 + 物料憑證年度 / 號碼 / 行項 + 關聯類型」，執行時只是把這筆關聯塞進一張**全域內部表 G_QAMBTAB**，真正寫進資料庫的動作是靠 **PERFORM SICHERN_QAMB ON COMMIT**——**ON COMMIT** 是 ABAP 語法糖，效果類似 **CALL FUNCTION ... IN UPDATE TASK**（都是「登記一段邏輯，等 **COMMIT WORK** 觸發時才真正執行」），只是這裡用在 **PERFORM** 層級而不是 **FM** 層級。**SICHERN_QAMB** 本體最後又呼叫了一支 Update FM **QEVA_MATERIALDOC_TO_LOT (IN UPDATE TASK)**，這才是真正寫入 **QAMB** 表的地方。這是 **if03** 要教的 **LUW** 概念在標準程式裡的活教材：**QAMB_COLLECT_RECORD** 呼叫完，**sy-subrc** 是 0，但資料庫裡其實還沒有這筆紀錄，要等到後續 **COMMIT WORK** 才會真正落地。

③ **STATUS_CHANGE_INTERN**——這不是 **QM** 專用 **FM**，是整個 **SAP** 系統共用的 **Status Management** 通用 **API**：

```
FUNCTION STATUS_CHANGE_INTERN
  IMPORTING
    VALUE(CHECK_ONLY) TYPE XFELD DEFAULT SPACE
    VALUE(CLIENT) LIKE SY-MANDT DEFAULT SY-MANDT
    VALUE(OBJNR) LIKE JSTO-OBJNR
    VALUE(ZEILE) LIKE MESG-ZEILE DEFAULT SPACE
    VALUE(SET_CHGKZ) LIKE JSTO-CHGKZ OPTIONAL
  EXPORTING
    VALUE(ERROR_OCCURRED) TYPE ANY
    VALUE(OBJECT_NOT_FOUND) TYPE ANY
```

```

VALUE(STATUS_INCONSISTENT) TYPE ANY
VALUE(STATUS_NOT_ALLOWED) TYPE ANY
TABLES
  STATUS LIKE JSTAT
EXCEPTIONS
  OBJECT_NOT_FOUND
  STATUS_INCONSISTENT
  STATUS_NOT_ALLOWED.

```

OBJNR (Object Number , **JSTO-OBJNR**) 是 SAP 通用狀態管理 (Status Management , **JSTO/JEST/JSTAT** 這組表) 的通用鍵值——不只檢驗批，生產訂單、維修通知單、任何「有系統狀態 / 使用者狀態」的物件都用同一套機制，檢驗批只是眾多可以掛狀態的物件類型之一。TABLES status LIKE jstat 吃的是一張「要設定的狀態清單」，JSTAT 結構只有兩個欄位：stat (狀態代碼) + **inact** (X = 取消這個狀態，空白 = 啟用這個狀態) 。

④ 反沖對照——不透過 **QM**、直接下 **MM** 庫存異動用的官方 **BAPI**：

```

FUNCTION bapi_goodsmvt_create
IMPORTING
  VALUE(goodsmvt_header) LIKE bapi2017_gm_head_01
  VALUE(goodsmvt_code) LIKE bapi2017_gm_code
  VALUE(testrun) LIKE bapi2017_gm_gen-testrun DEFAULT space
  ...
EXPORTING
  VALUE(goodsmvt_headret) LIKE bapi2017_gm_head_ret
  VALUE(materialdocument) TYPE bapi2017_gm_head_ret-mat_doc
  VALUE(matdocumentyear) TYPE bapi2017_gm_head_ret-doc_year
TABLES
  goodsmvt_item LIKE bapi2017_gm_item_create
  ...
  return LIKE bapiret2
  ...

```

```

FUNCTION bapi_goodsmvt_cancel
IMPORTING
  VALUE(materialdocument) TYPE bapi2017_gm_head_02-mat_doc
  VALUE(matdocumentyear) TYPE bapi2017_gm_head_02-doc_year
  ...
TABLES
  return LIKE bapiret2
  ...

```

跟 **BAPI_INSPLLOT_SETUSAGEDECISION** / 情境 B 那四支比起來，**BAPI_GOODSMVT_CREATE** / **BAPI_GOODSMVT_CANCEL** 明顯「規矩」很多：**CREATE** 回傳的 **materialdocument** / **matdocumentyear** 正好就是 **CANCEL** 要吃的 **IMPORTING** 參數 (沖銷一定要指名沖哪一張憑證)，錯誤一律走 TABLES **return LIKE bapiret2** 結構化訊息，沒有任何依賴 Function Group 全域變數的設計——這正是正式 **Released BAPI** 的介面

紀律跟情境 B 那組「內部拼裝」FM 的差異所在，也呼應 if01 教的「BAPI vs 一般 FM」判準。有趣的是，BAPI_GOODSMVT_CANCEL 原始碼裡自己也是 PERFORM mb_post_goods_movement（呼叫同名的內部子程式，不是本題查到的那支獨立 FM）完成實際過帳——連正式 BAPI 骨子裡也是疊在同一套底層過帳邏輯上，只是 BAPI 這層把介面包乾淨了。

學習目標

- 能用 ADT quickSearch（或 SE37 / BAPI Explorer）確認一個 FM / BAPI 是否存在於系統中，並讀出它的真實介面定義
- 分清楚「下使用決策同時過帳」（BAPI_INSPLT_SETUSAGEDECISION + ud_stock_posting）跟「單獨補過帳」（MB_CREATE_GOODS_MOVEMENT → MB_POST_GOODS_MOVEMENT → QAMB_COLLECT_RECORD → STATUS_CHANGE_INTERN）是兩條不同的路徑，不能混用
- 理解「呼叫成功（sy-subrc = 0）」不等於「資料庫已經真的寫入」——QAMB_COLLECT_RECORD 的 PERFORM ... ON COMMIT 是活教材
- 理解 MB_CREATE_GOODS_MOVEMENT / MB_POST_GOODS_MOVEMENT 這種「靠 Function Group 全域變數溝通」的舊式介面設計，跟 BAPI「每支介面自帶所有輸入輸出」的設計哲學差在哪裡
- 能看懂 STATUS_CHANGE_INTERN 是通用狀態管理 API，不是 QM 專屬

事前準備

不需要新建任何 SAP 物件——這題全部使用 SAP 標準物件，已於本次連線（client 130，2026-07-21）用 sap-adt MCP 的 sap_get_source 直接讀取原始碼驗證存在與介面簽章：

FM / BAPI	Function Group	套件
BAPI_INSPLT_SETUSAGEDECISION	2045	QL
BAPI_INSPLT_GETDETAIL	2045	QL
MB_CREATE_GOODS_MOVEMENT	MBWL	MB
MB_POST_GOODS_MOVEMENT	MBWL	MB
QAMB_COLLECT_RECORD	QAMB	QL
STATUS_CHANGE_INTERN	BSVA	BSV
BAPI_GOODSMVT_CREATE	MB_BUS2017	MB
BAPI_GOODSMVT_CANCEL	MB_BUS2017	MB

題目需求

1. 情境 A 分析：對照 BAPI_INSPLT_SETUSAGEDECISION 的原始碼，指出「呼叫成功後該做什麼、失敗該做什麼」這件事是從哪裡得到證據的（不是猜的，是原始碼裡的哪一段）。
2. 情境 B 呼叫順序圖：畫出（或寫出）MB_CREATE_GOODS_MOVEMENT → MB_POST_GOODS_MOVEMENT → QAMB_COLLECT_RECORD → STATUS_CHANGE_INTERN 的呼叫順序，並在每一步標注「這一步的資料現在存在哪裡」（記憶體全域變數 / 全域內部表 / 資料庫）。
3. 欄位對照：從 Lecture 列出的 BAPI2045UD 欄位表，指出哪一個欄位是情境 A「同時過帳」的開關。
4. 反沖配對：說明 BAPI_GOODSMVT_CREATE 的哪兩個 EXPORTING 參數，會原封不動變成 BAPI_GOODSMVT_CANCEL 的 IMPORTING 參數。

5. 介面骨架設計（只設計，不建物件、不執行）：設計一個 `ZCL_IF02_INSPL0T_POSTING` 類別的方法簽章草案，包含 `post_with_ud`（對應情境 A，包一層 `BAPI_INSPL0T_SETUSAGEDECISION` + `BAPI_TRANSACTION_COMMIT/ROLLBACK`）與 `post_open_stock`（對應情境 B，包一層四支 FM 序列 + `COMMIT WORK/ROLLBACK WORK`）兩個方法，只要簽章與方法內的呼叫順序註解，不用寫完整實作。

團隊實務備註

- 找到這八支物件全靠 `.claude/rules/sap-adt-mcp.md` 第 2 節記載的 **ADT quickSearch workaround**，比原本設想的「BAPI Explorer 交易碼點來點去」快很多，對於「已經知道名字、只是要確認存不存在 + 讀介面」的情境特別好用；`sap_search_object` 這個 MCP 工具本身依然是壞的（第 2 節記載的已知限制，這次沒有重新測試，直接跳過用 workaround）。
- 這題示範的「往回追」方法論，實際上只做到「查名字 + 讀介面」這一步，若要百分之百確認「QA12 的『Inspection lot stock』頁籤按 Save 時，真的是照這個順序呼叫這四支 FM」，需要 README 記載的 SAT / ST05 / Debugger 這幾招在系統上實際操作驗證，這份講義只能做到「這四支 FM 存在、簽章長這樣、彼此的資料流關係看原始碼推得出來」，還沒有做到「亲眼看到 QA12 真的這樣呼叫」的程度——出題時若要百分之百坐實，建議下次連線時補做這一步。
- `MB_CREATE_GOODS_MOVEMENT` 的原始碼開頭有一行 `def_break 'MB_CREATE_GOODS_MOVEMENT'`。——這是 SAP 標準程式碼裡常見的「條件式中斷點」寫法（`def_break / DEFINE BREAK` 是 SAP 內部除錯輔助巨集，符合特定使用者設定才會真的中斷），順便讓學員認識這個常在標準程式碼看到、但基礎課沒教過的除錯慣例。

思考題

1. 如果今天寫一支自己的整合程式，同時需要「情境 A」（有些檢驗批一次判定就過帳）跟「情境 B」（有些檢驗批判定後才補過帳），程式該怎麼設計才能兩種情境共用一套邏輯，而不是複製貼上兩份呼叫序列？（提示：兩者最後都會走到「物料憑證 + QAMB 關聯 + 狀態更新」這個結果，但情境 A 是 SAP 內部封裝好的、情境 B 要自己組——可以思考「要不要乾脆都走情境 B 那組底層 FM，情境 A 的『同時』只是省了呼叫者自己判斷的力氣」）
2. `QAMB_COLLECT_RECORD` 用 `PERFORM ... ON COMMIT`，`BAPI_INSPL0T_SETUSAGEDECISION` 內部用的是任何的 `CALL FUNCTION`（沒有 `IN UPDATE TASK`）。這是否代表 `BAPI_INSPL0T_SETUSAGEDECISION` 本身完全不涉及「延遲寫入、COMMIT 時才真正生效」的機制？（提示：不代表——`QEVC_PROCESS_AUTO_UD` 這一層以下的實作細節這題沒有深入追，只能確定「BAPI 呼叫端本身不做 `COMMIT WORK`」這個外部可觀察的事實，內部是否也用了類似 `ON COMMIT / IN UPDATE TASK` 的機制，需要再往下追一層才能確定，這也是「讀原始碼能確認到什麼程度」的一個誠實邊界）
3. `STATUS_CHANGE_INTERN` 是通用狀態管理 API，`OBJNR` 決定要改哪個物件的狀態。如果要改一個檢驗批的狀態，`OBJNR` 這個值應該從哪裡取得？（提示：SAP 通用狀態管理的物件號碼通常存在該業務物件自己的主檔資料裡，例如檢驗批表 `QALS` 應該就有一個對應欄位存放它自己的 `OBJNR`，這題沒有實際查證這個欄位名稱，出題時建議用 `sap_get_source / sap_object_structure` 讀一下 `QALS` 表結構確認）

答案

本題不建立任何新 SAP 物件；所有 FM / BAPI 介面定義均直接來自本次連線系統（client 130，2026-07-21）用 `sap-adt` MCP `sap_get_source` 讀出的真實原始碼，未經改寫（僅節錄與省略部分次要參數 / 實作細節，節錄處已標示）。